Лабораторая работа 2

Основные модели: SIR и Лотки-Вольтерры

Закиров Нурислам Дамирович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Создание рабочего каталога для кода и установка необходимых пакетов 9
4.2 Выполнение предложенного кода SIR. 9
4.3 Выполнение предложенного кода LV. 12
4.4 Преобразование кода модели SIR в литературный стиль. 14
4.5 Генерация из литературного кода модели SIR в jupyter notebook. . . . 15
4.6 Генерация из литературного кода модели SIR в документацию в формате Quarto. 16
4.7 Генерация из литературного кода модели SIR в чистый код. 17
4.8 Преобразование кода модели LV в литературный стиль. 18
4.9 Генерация из литературного кода модели LV в jupyter notebook. . . . 19
4.10 Генерация из литературного кода модели LV в документацию в формате Quarto. 20
4.11 Генерация из литературного кода модели LV в чистый код. 21
4.12 Добавление в код в литературном стиле вычисление для набора параметров. 23
4.13 Генерация из литературного кода с параметрами в чистый код;jupyter notebook;документацию в формате Quarto. 24

5 Выводы **27**

Список литературы **28**

Список иллюстраций

4.1 Рабочий каталог для кодов . . . 9
4.2 Предложенный код модели SIR . . . 10
4.3 Попытка запуска модели SIR . . . 10
4.4 Успешная дозагрузка пакетов для запуска модели SIR . . . 11
4.5 Успешное выполнение модели SIR . . . 11
4.6 Успешное выполнение модели SIR . . . 11
4.7 Предложенный код модели LV . . . 12
4.8 Попытка запуска модели LV . . . 12
4.9 Успешное выполнение модели LV . . . 13
4.10 Успешное выполнение модели LV . . . 13
4.11 Создание файла формата qmd . . . 14
4.12 Оформление код модели SIR в литературный стиль . . . 14
4.13 Генерация jupyter notebook . . . 15
4.14 Запуск jupyter notebook . . . 15
4.15 Результат генерации литературного кода в jupyter notebook . . . 16
4.16 Генерация литературного кода в документацию в формате Quarto . . . 16
4.17 Созданный файл sir_ode.html . . . 17
4.18 Результат генерации литературного кода в документацию в формате Quarto . . . 17
4.19 Генерация литературного кода в документацию в формате Quarto . . . 17
4.20 Результат генерации литературного кода в чистый код . . . 18
4.21 Создание файла формата qmd . . . 18
4.22 Оформление код модели LV в литературный стиль . . . 19
4.23 Генерация jupyter notebook . . . 19
4.24 Результат генерации литературного кода в jupyter notebook . . . 20
4.25 Генерация литературного кода в документацию в формате Quarto . . . 20
4.26 Результат генерации литературного кода в документацию в формате Quarto . . . 21
4.27 Генерация литературного кода в документацию в формате Quarto . . . 22
4.28 Сгенерированный графики . . . 22
4.29 Каталог . . . 23
4.30 Код вычисление для набора параметров . . . 23
4.31 Каталог . . . 25
4.32 Результат генерации в формате html . . . 25
4.33 График результата генерации . . . 26

Список таблиц

1 Цель работы

Изучение и программная реализация базовых математических моделей: модели эпидемии SIR и модели хищник-жертва Лотки-Вольтерры. Освоение методов программирования в литературном стиле.

2 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:чистый код;jupyter notebook;документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:чистый код;jupyter notebook;документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

В данной лабораторной работе рассматриваются две фундаментальные математические модели.

3.0.1 1. Модель SIR

Модель SIR есть классическая и фундаментальная математическая модель эпидемиологии, описывающая распространение инфекционного заболевания в закрытой популяции. Модель делит всю популяцию на три взаимосвязанные группы (компартменты):

- **S** (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться.
- **I** (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию.
- **R** (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет.

Динамика модели описывается тремя уравнениями:

$$\frac{dS}{dt} = -\beta IS/N$$

$$\frac{dI}{dt} = \beta IS/N - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

3.0.2 2. Модель Лотки-Вольтерры

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв. Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы.

Система состоит из двух дифференциальных уравнений:

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

4 Выполнение лабораторной работы

4.1 Создание рабочего каталога для кода и установка необходимых пакетов

Для начала выполнения лабораторной работы, создаем папки scripts, notebooks, docx. На первом этапе рабочий каталог будет выглядить таким образом. В дальнейшем, в ходе выполнения работы, структура каталога будет изменена. Также хочу обратить внимание на то, что все пакеты были заранее установлены на мою виртуальную мишину, поэтому данный этап мы пропускаем.(рис. 4.1).

Рисунок 4.1: Рабочий каталог для кодов

4.2 Выполнение предложенного кода SIR.

При помощи nano sir_ode.jl создаем файл, в который мы скопировали предложенный код из инструкции по выполнению лаболаторной работы.(рис. 4.2).

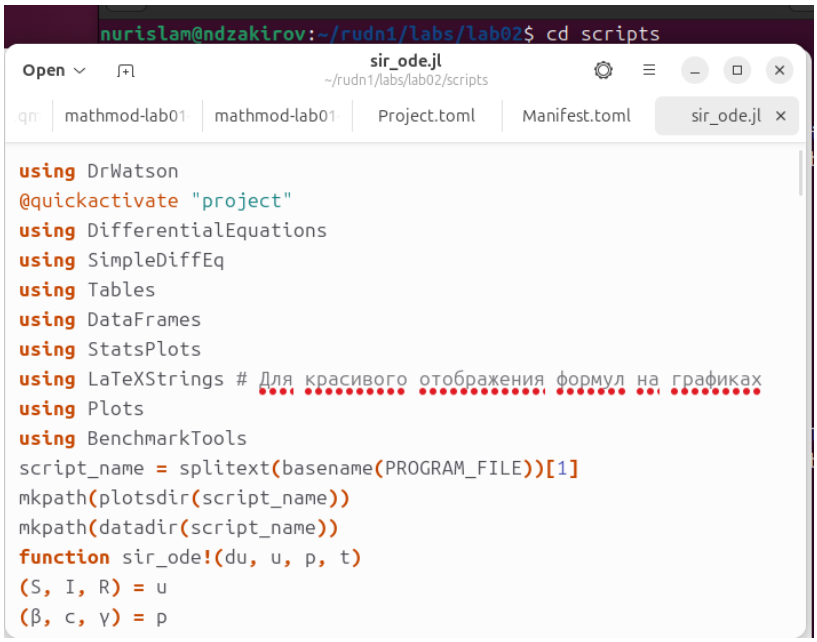

Рисунок 4.2: Предложенный код модели SIR

Далее я пытаюсь запустить данный код при помощи julia . Но, как оказалось, не все пакеты были доустановленны, поэтому дожидаемся установки недостающих пакетов julia(рис. 4.3).

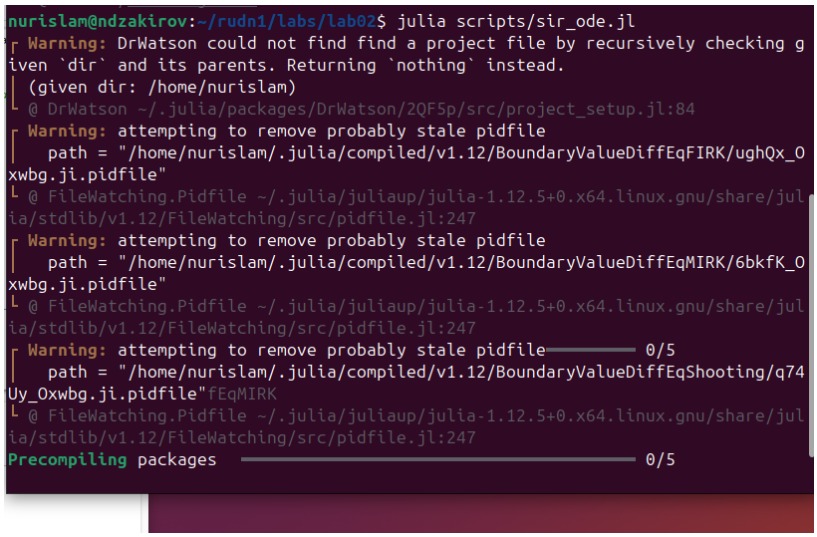

Рисунок 4.3: Попытка запуска модели SIR

Как мы видим, все недостающие пакеты были установлены в полном

объеме.(рис. 4.4).

```
nurislam@ndzakirov:~/rudn1/labs/lab02$ julia -e 'using Pkg; Pkg.add("BenchmarkTo
ols")'
   Resolving package versions...
    Updating `~/.julia/environments/v1.12/Project.toml`
  [6e4b80f9] + BenchmarkTools v1.6.3
    Updating `~/.julia/environments/v1.12/Manifest.toml`
  [6e4b80f9] + BenchmarkTools v1.6.3
  [9abbd945] + Profile v1.11.0
Precompiling packages finished.
  1 dependency successfully precompiled in 7 seconds. 513 already precompiled.
nurislam@ndzakirov:~/rudn1/labs/lab02$
```

Рисунок 4.4: Успешная дозагрузка пакетов для запуска модели SIR

После установки недостающик пакетов, мы успешно выполнили предложенный код из инструкции по выполнению лаболаторной работы.(рис. 4.5).

```
nurislam@ndzakirov:~/rudn1/labs/lab02$ julia scripts/sir_ode.jl
┌ Warning: DrWatson could not find find a project file by recursively checking g
iven `dir` and its parents. Returning `nothing` instead.
│ (given dir: /home/nurislam)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
┌ Warning: Using the standard Julia project.
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:30
┌ Warning: Using the standard Julia project.
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:30
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R0 = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 =

0.0
```

Рисунок 4.5: Успешное выполнение модели SIR

Также важно посмотреть на результаты Анализа выполнения sir_ode.jl.(рис. 4.6).

```
Бенчмарк решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
 - Порог коллективного иммунитета: 50.0%
 - Теоретический пик при S/N = 1/R0 = 0.5
nurislam@ndzakirov:~/rudn1/labs/lab02$
```

Рисунок 4.6: Успешное выполнение модели SIR

4.3 Выполнение предложенного кода LV.

При помощи nano lv_ode.jl создаем файл, в который мы скопировали предложенный код модели Лотки-Вольтерры из инструкции по выполнению лаболаторной работы.(рис. 4.7).

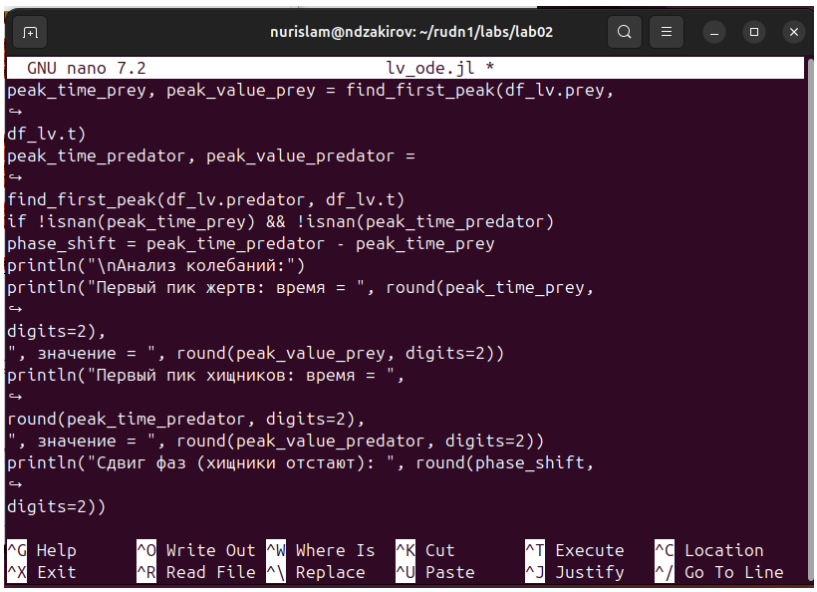

Рисунок 4.7: Предложенный код модели LV

Далее я пытаюсь запустить данный код при помощи julia . Но, как оказалось, опять не все пакеты были доустановленны, поэтому дожидаемся установки недостающих пакетов julia(рис. 4.8).

Рисунок 4.8: Попытка запуска модели LV

После установки недостающик пакетов, мы успешно выполнили предложенный код модели Лотки-Вольтерры из инструкции по выполнению лаболаторной работы.(рис. 4.9).

```
nurislam@ndzakirov:~/rudn1/labs/lab02$ julia scripts/lv_ode.jl
┌ Warning: DrWatson could not find find a project file by recursively checking given `dir` and its parents. Returning `n
othing` instead.
│ (given dir: /home/nurislam)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
┌ Warning: Using the standard Julia project.
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:30
┌ Warning: Using the standard Julia project.
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:30
┌ Warning: Verbosity toggle: dense_output_saveat
│  Dense output is incompatible with saveat. Please use the SavingCallback from the Callback Library to mix the two beha
viors.
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/tusSd/src/solve.jl:153
============================================================
Модель Лотки-Вольтерры (хищник-жертва)
============================================================

Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0

Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0
```

Рисунок 4.9: Успешное выполнение модели LV

Также, считаю важным посмотреть и на результаты Анализа выполнения lv_ode.jl.(рис. 4.10).

```
Модель Лотки-Вольтерры (хищник-жертва)
============================================================

Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0

Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0

Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени

============================================================
Анализ результатов
============================================================

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
```

Рисунок 4.10: Успешное выполнение модели LV

4.4 Преобразование кода модели SIR в литературный стиль.

Слудующим важным этапы нашей работы является создание литературного стиля. Для начал мы создадим файл формата qmd для модели sir при помощи команды nano(рис. 4.11).

Рисунок 4.11: Создание файла формата qmd

Далее оформляем наш код в литературном стиле, разделяя коментарии от кода. Код мы окружаем метками “`{julia}`, а коментарии оставляем как есть, они автоматически считываются форматом markdown. (рис. 4.12).

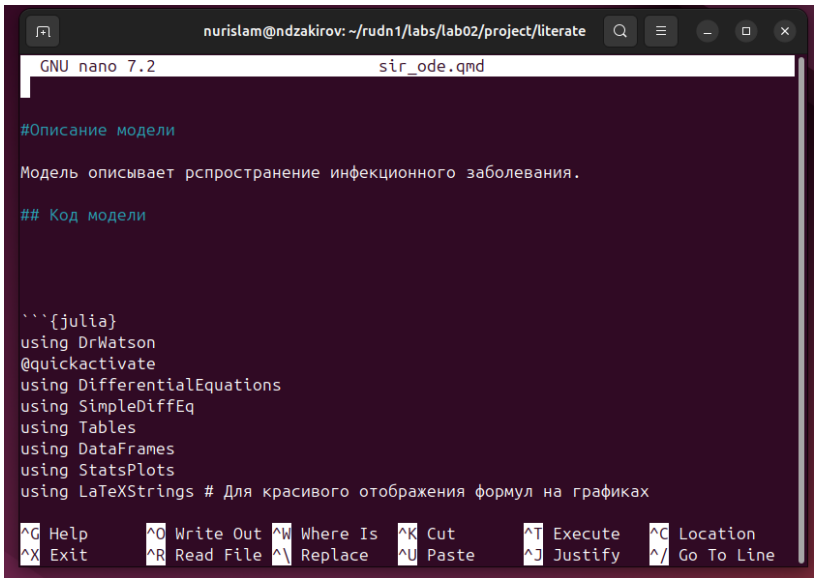

Рисунок 4.12: Оформление код модели SIR в литературный стиль

4.5 Генерация из литературного кода модели SIR в jupyter notebook.

Далее генерируем jupyter notebook, через quarto render. (рис. 4.13).

```
nurislam@ndzakirov:~/rudn1/labs/lab02/project/literate$ quarto render sir_ode.qm
d --to ipynb

Starting julia-1.12 kernel...Done

Executing 'sir_ode.quarto_ipynb'
  Cell 1/24: ''...Done
  Cell 2/24: ''...Done
  Cell 3/24: ''...Done
  Cell 4/24: ''...Done
  Cell 5/24: ''...Done
  Cell 6/24: ''...Done
  Cell 7/24: ''...Done
  Cell 8/24: ''...Done
  Cell 9/24: ''...Done
  Cell 10/24: ''...Done
  Cell 11/24: ''...Done
  Cell 12/24: ''...Done
  Cell 13/24: ''...Done
  Cell 14/24: ''...Done
  Cell 15/24: ''...Done
  Cell 16/24: ''...Done
  Cell 17/24: ''...Done
  Cell 18/24: ''...Done
```

Рисунок 4.13: Генерация jupyter notebook

Далее запускаем jupyter notebook, через терминал. (рис. 4.14).

```
nurislam@ndzakirov:~/rudn1/labs/lab02/project/literate$ jupyter notebook
[I 11:40:19.020 NotebookApp] Serving notebooks from local directory: /home/nuris
lam/rudn1/labs/lab02/project/literate
[I 11:40:19.021 NotebookApp] Jupyter Notebook 6.4.12 is running at:
[I 11:40:19.021 NotebookApp] http://localhost:8888/?token=62a075af2424f098434dd9
9fad22dd241052302e6a1402c6
[I 11:40:19.021 NotebookApp]  or http://127.0.0.1:8888/?token=62a075af2424f09843
4dd99fad22dd241052302e6a1402c6
[I 11:40:19.021 NotebookApp] Use Control-C to stop this server and shut down all
 kernels (twice to skip confirmation).
[C 11:40:19.271 NotebookApp]

    To access the notebook, open this file in a browser:
        file:///home/nurislam/.local/share/jupyter/runtime/nbserver-3494-open.ht
ml
    Or copy and paste one of these URLs:
        http://localhost:8888/?token=62a075af2424f098434dd99fad22dd241052302e6a1
402c6
     or http://127.0.0.1:8888/?token=62a075af2424f098434dd99fad22dd241052302e6a1
402c6
```

Рисунок 4.14: Запуск jupyter notebook

После чего просматриваем результат генерации и подтверждаем его корректную работу. (рис. 4.15).

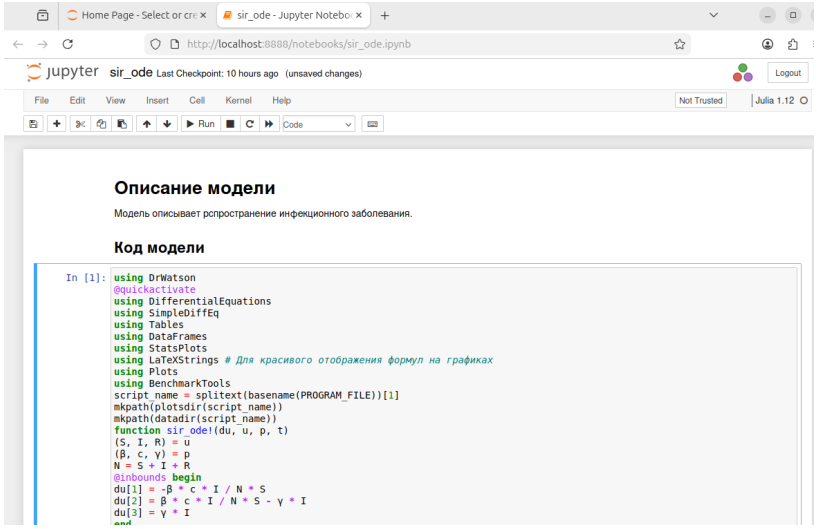

Рисунок 4.15: Результат генерации литературного кода в jupyter notebook

4.6 Генерация из литературного кода модели SIR в документацию в формате Quarto.

Таким же образом, при помощи quarto render конвертируем наш код формата qmd в формат html, для того, чтобы создать документацию в формате Quarto с разделением на блоки кода и комментариев. (рис. 4.16).

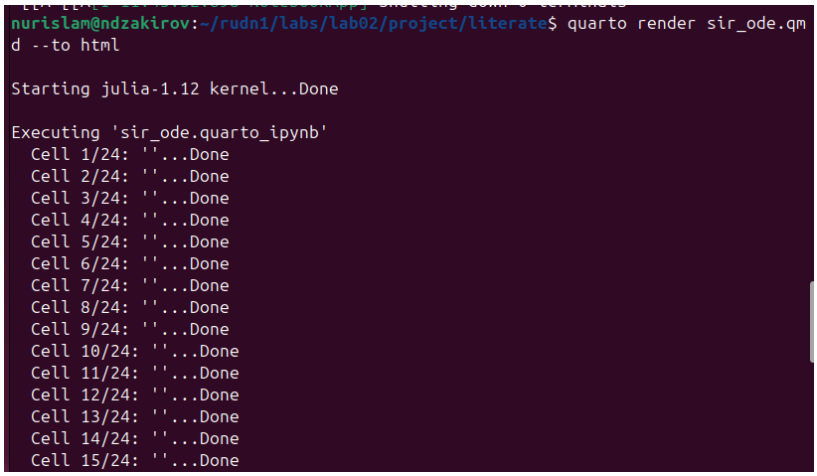

Рисунок 4.16: Генерация литературного кода в документацию в формате Quarto

Далее, при помощи ls, мы видим, что генерация завершилась и в нашей дериктории создался файл sir_ode.html. (рис. 4.17).

```
nurislam@ndzakirov:~/rudn1/labs/lab02/project/literate$ ls
data            plots          sir_ode_files  sir_ode.ipynb
Manifest.toml   Project.toml   sir_ode.html   sir_ode.qmd
```

Рисунок 4.17: Созданный файл sir_ode.html

Запускаем файл sir_ode.html и тоже убеждаемся в корректной работе генерации. (рис. 4.18).

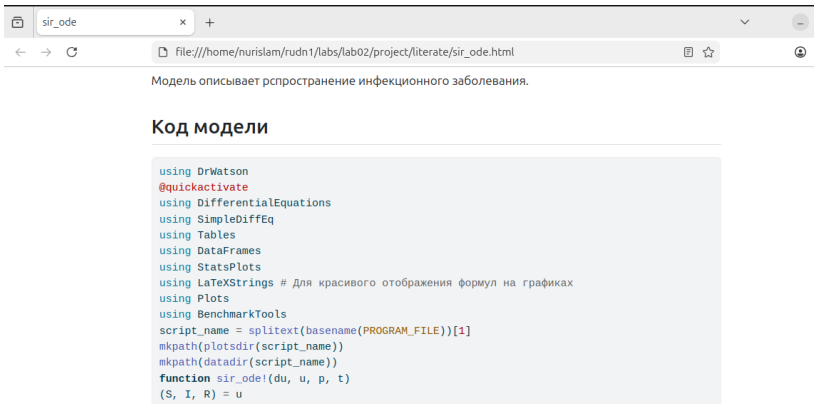

Рисунок 4.18: Результат генерации литературного кода в документацию в формате Quarto

4.7 Генерация из литературного кода модели SIR в чистый код.

Далее при помощи nbconvert конвертируем наш код формата qmd в формат jl, для того, чтобы создать чистый код. (рис. 4.19).

```
nurislam@ndzakirov:~/rudn1/labs/lab02/project/literate$ python3 -m nbconvert --t
o script sir_ode.ipynb
[NbConvertApp] Converting notebook sir_ode.ipynb to script
[NbConvertApp] Writing 5728 bytes to sir_ode.jl
nurislam@ndzakirov:~/rudn1/labs/lab02/project/literate$
```

Рисунок 4.19: Генерация литературного кода в документацию в формате Quarto

Запускаем файл sir_ode.jl и снова убеждаемся в корректной работе генерации. (рис. 4.20).

Рисунок 4.20: Результат генерации литературного кода в чистый код

4.8 Преобразование кода модели LV в литературный стиль.

Таким же образом мы создадим файл формата qmd для модели LV при помощи команды nano(рис. 4.21).

Рисунок 4.21: Создание файла формата qmd

Далее оформляем наш код в литературном стиле, разделяя коментарии от кода. Код мы окружаем метками «`{julia}`», а коментарии оставляем как есть, они автоматически считываются форматом markdown. (рис. 4.22).

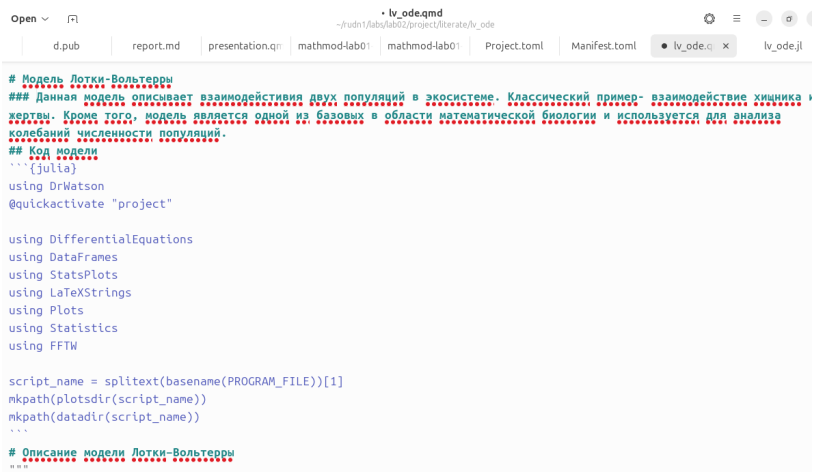

Рисунок 4.22: Оформление код модели LV в литературный стиль

4.9 Генерация из литературного кода модели LV в jupyter notebook.

Далее генерируем jupyter notebook, через quarto render. (рис. 4.23).

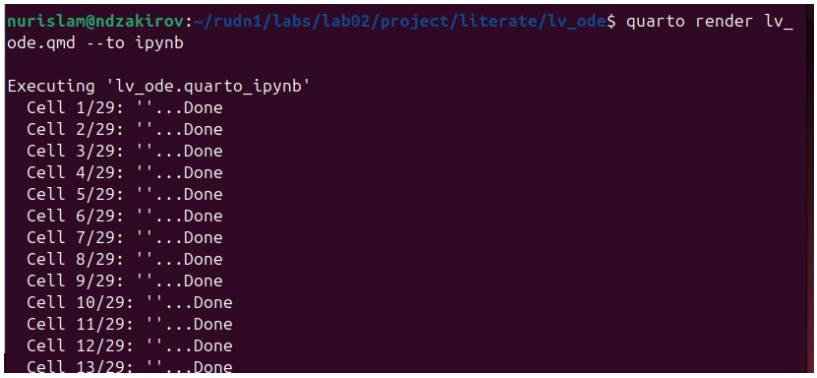

Рисунок 4.23: Генерация jupyter notebook

После чего просматриваем результат генерации и подтверждаем его корректную работу. (рис. 4.24).

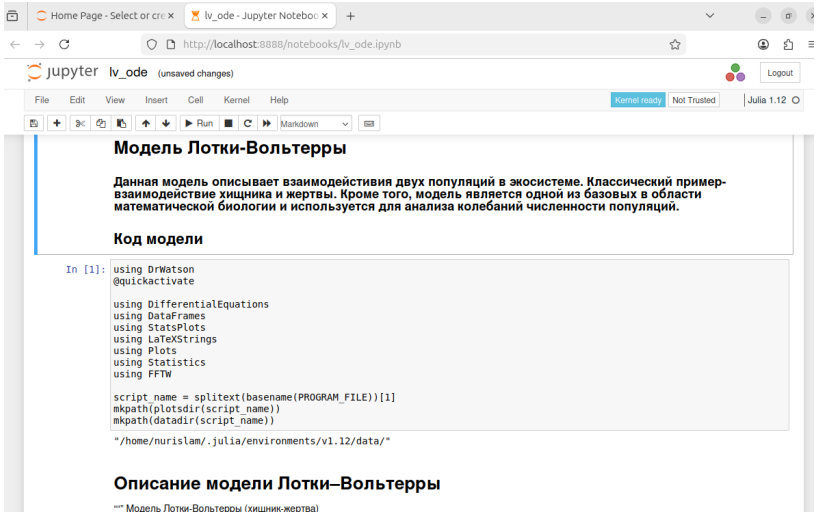

Рисунок 4.24: Результат генерации литературного кода в jupyter notebook

4.10 Генерация из литературного кода модели LV в документацию в формате Quarto.

Таким же образом, при помощи quarto render конвертируем наш код формата qmd в формат html, для того, чтобы создать документацию в формате Quarto с разделением на блоки кода и комментариев. (рис. 4.25).

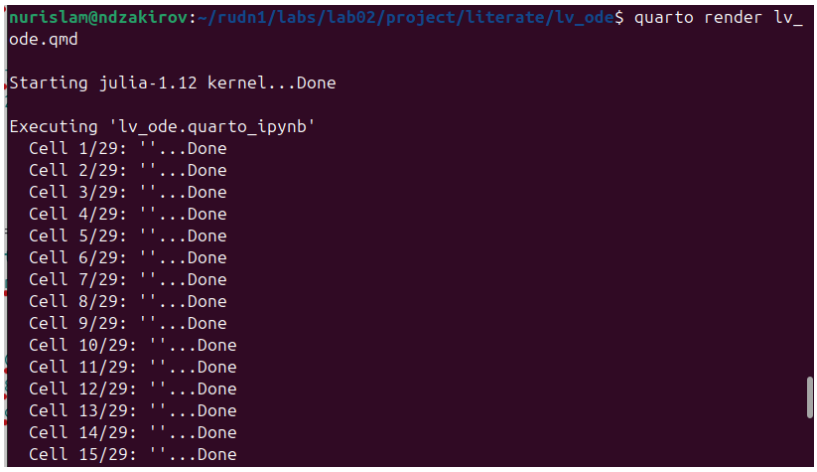

Рисунок 4.25: Генерация литературного кода в документацию в формате Quarto

Запускаем файл lv_ode.html и тоже убеждаемся в корректной работе генерации.

(рис. 4.26).

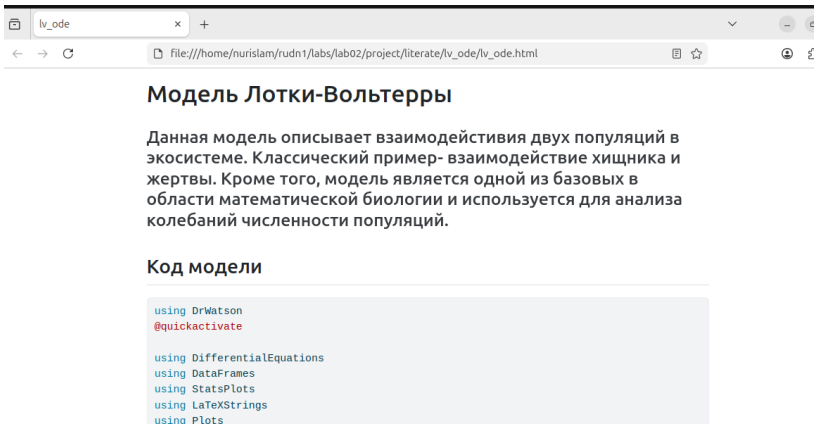

Рисунок 4.26: Результат генерации литературного кода в документацию в формате Quarto

4.11 Генерация из литературного кода модели LV в чистый код.

Далее при помощи nbconvert конвертируем наш код формата qmd в формат jl, для того, чтобы создать чистый код и снова убеждаемся в корректной работе генерации. (рис. 4.27).

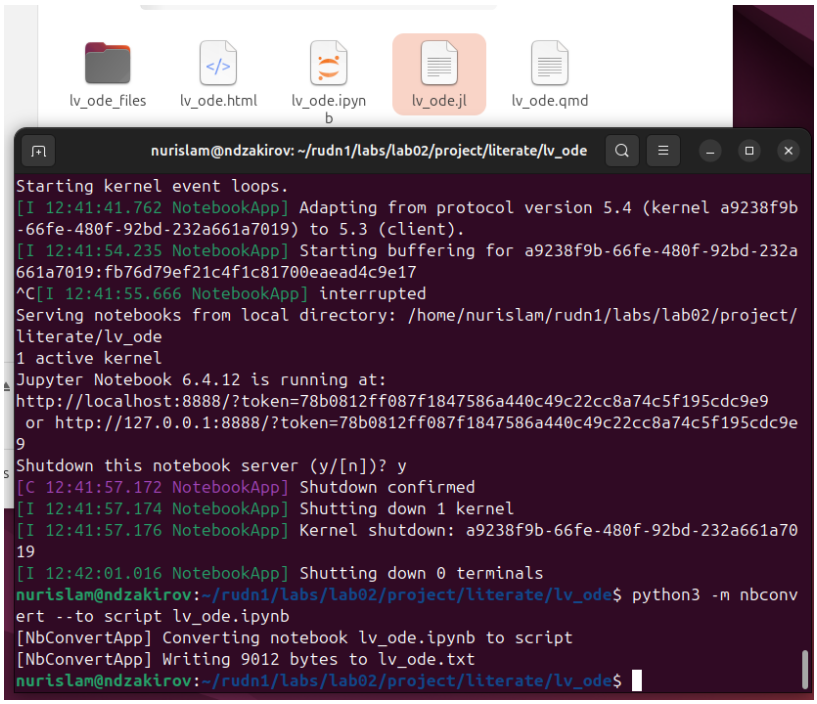

Рисунок 4.27: Генерация литературного кода в документацию в формате Quarto

Также стоит обратить внимание что мы поменяли структуру каталогов, теперь у нас в папке project, есть папка literate, в которой лежат папки sir_ode и lv_ode. И также продемонстрирую создавшуюся папку /literate/lv_ode/Plots с изображениями всех нарисованных графиков в ходе выполнения програмного кода (рис. 4.28).

Рисунок 4.28: Сгенерированный графики

И также подтвердим выполнение данного этапа скриншотом нашего каталога, где создались все необходимые файлы (рис. 4.29).

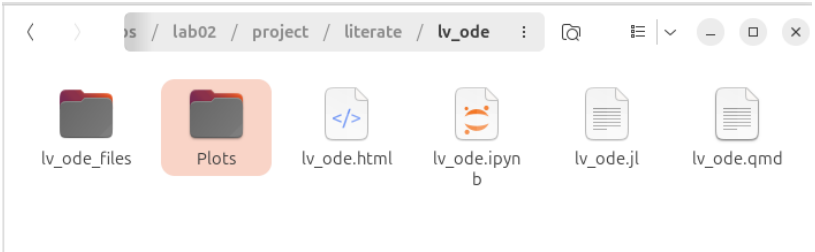

Рисунок 4.29: Каталог

4.12 Добавление в код в литературном стиле вычисление для набора параметров.

Создаем sir_ode(edited), куда мы записываем sir_ode.qmd с добавлением вычисления для набора параметров, продемонстрированном на (рис. 4.30).

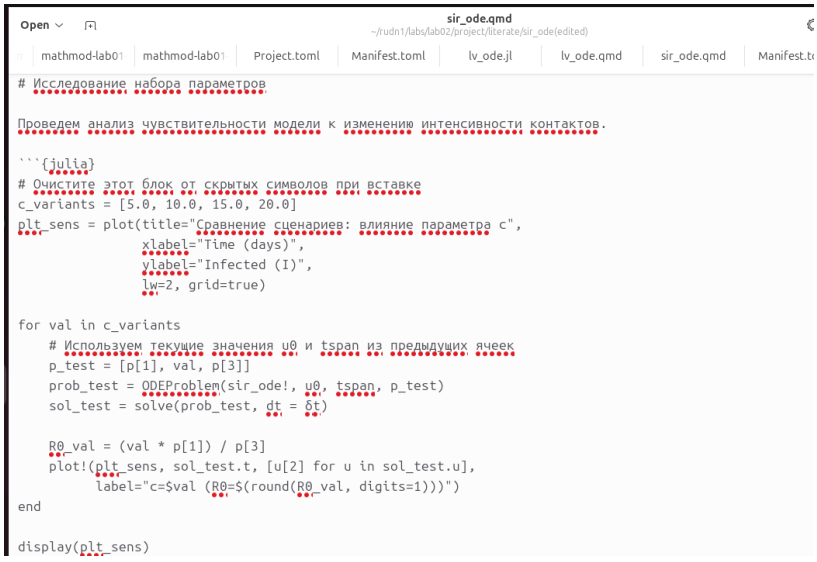

Рисунок 4.30: Код вычисление для набора параметров

Код вычисление для набора параметров:

```
c_variants = [5.0, 10.0, 15.0, 20.0]
plt_sens = plot(title="[redacted]: [redacted] [redacted] c",
                xlabel="Time (days)",
```

```
                ylabel="Infected (I)",
                lw=2, grid=true)

for val in c_variants
    p_test = [p[1], val, p[3]]
    prob_test = ODEProblem(sir_ode!, u0, tspan, p_test)
    sol_test = solve(prob_test, dt = Δt)

    R0_val = (val * p[1]) / p[3]
   plot!(plt_sens, sol_test.t, [u[2] for u in sol_test.u],
         label="c=$val (R0=$(round(R0_val, digits=1)))")
end

display(plt_sens)
```

4.13 Генерация из литературного кода с параметрами в чистый код;jupyter notebook;документацию в формате Quarto.

Таким же образом генерируем все требуемые форматы файла такими же методами, которые я описал выше. В результате мы получаем каталог, со всеми форматами, которые были запрошены в тз (рис. 4.31).

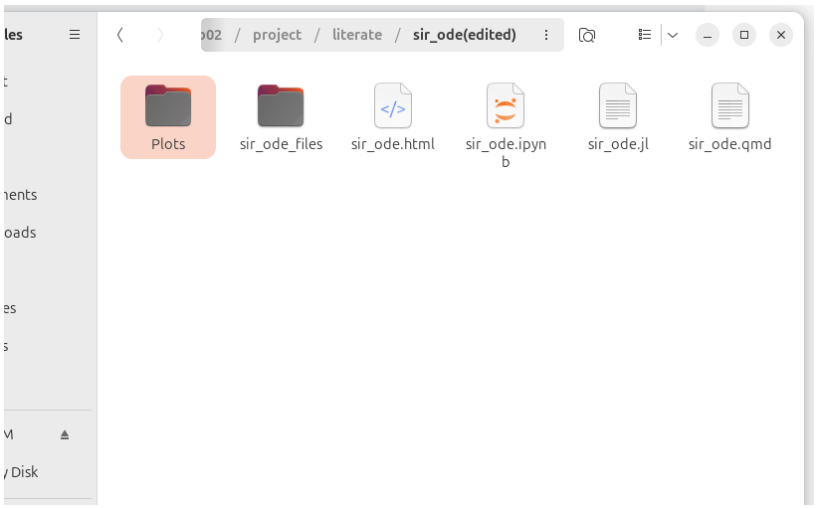

Рисунок 4.31: Каталог

Продемонстрируем результат работы нашей генерации (рис. 4.32).

Исследование набора параметров

Проведем анализ чувствительности модели к изменению интенсивности контактов.

```
# Очистите этот блок от скрытых символов при вставке
c_variants = [5.0, 10.0, 15.0, 20.0]
plt_sens = plot(title="Сравнение сценариев: влияние параметра c",
                xlabel="Time (days)",
                ylabel="Infected (I)",
                lw=2, grid=true)

for val in c_variants
    # Используем текущие значения u0 и tspan из предыдущих ячеек
    p_test = [p[1], val, p[3]]
    prob_test = ODEProblem(sir_ode!, u0, tspan, p_test)
    sol_test = solve(prob_test, dt = δt)

    R0_val = (val * p[1]) / p[3]
    plot!(plt_sens, sol_test.t, [u[2] for u in sol_test.u],
          label="c=$val (R0=$(round(R0_val, digits=1)))")
end

display(plt_sens)
```

Рисунок 4.32: Результат генерации в формате html

Также продемонстрируем график результата работы нашей генерации (рис. 4.33).

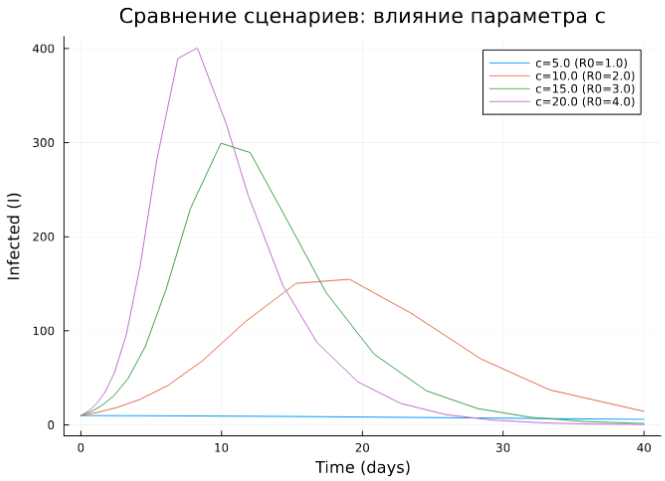

Рисунок 4.33: График результата генерации

5 Выводы

В ходе выполнения лабораторной работы я успешно изучил и смоделировал динамику систем по моделям SIR и Лотки-Вольтерры. Я освоил принципы литературного программирования, автоматизировал генерацию кода, Jupyter-ноутбуков и документации Quarto, а также научился исследовать влияние изменения параметров на поведение математических моделей.

Список литературы